Day 2: Math and Modules

For when you need a bit more firepower

2.1: Basic Math

To no one's surprise, you can do math in Python! Some of the basic operators (symbols that let you do an operation) are shown below:

Operator	What It Does
`+`	Addition
`-`	Subtraction
`*`	Multiplication
`/`	Division
`//`	Integer Divison
`%`	Modulus
`**`	Power (Exponent)

Below is a code snippet that shows all of them in use. I'll explain them more in detail in a bit:

```
print("25 + 5 =", 25 + 5)
print("25 - 5 =", 25 - 5)
print("25 * 5 =", 25 * 5)
print("25 / 5 =", 25 / 5)
print("25 // 5 =", 25 // 5)
print("25 % 5 =", 25 % 5)
print("25 ** 5 =", 25 ** 5)
```

```
25 + 5 = 30
25 - 5 = 20
25 * 5 = 125
25 / 5 = 5.0
25 // 5 = 5
25 % 5 = 0
25 ** 5 = 9765625
```

2.1.1: Division versus Integer Division

One thing you may have noticed is that there are *two* operators for division! Data types are covered in a separate day, but the important difference between `/` and `//` is the output. If you divide two integers with `/`, you will *always* get a decimal point. However, if you divide two integers with `//`, you will get only the *whole* part of the result:

```
print(25 / 6)
print(25 // 6)
```

```
4.166666666666667
4
```

`25 / 6` if evaluated using long divison yields you 4 r 1. With normal division, you get `4 + 1/6`, or `4.16...7` as seen in the output. With integer division, the remainder is discarded entirely.

> ⚠️ **Watch out — Do not confuse these two!**
>
> Using the wrong one can skew your results and cause you to have incorrect final outputs. Although Python tries its best to reflect how we do calculations, some things that Python does go against our typical instinct.

2.1.2: Modulus

Modulus is just means "give me the remainder from division". Using the `25` divided by `6` example from above, we should expect `1` as the output if we do modulus:

```
print(25 % 6)
```

```
1
```

The same concept applies to decimal numbers:

```
print(25 % 3.5)
```

```
0.5
```

`3.5` goes into `25` seven times (`3.5 * 7 = 24.5`), leaving `0.5` as the remainder. It's very useful for figuring out what is *leftover* (e.g., converting minutes into days, hours, and minutes).

2.1.3: Order of Operations

Order of operations in Python is the same as BEDMAS/PEDMAS/whatever acronym you use. Modulus and integer divison are in the same category as division and multiplication.

> 📝 **Note — There are more operators!**
>
> These are traditionally the basic operators for math. However, there are even more math operators and just more operators in general! You'll learn more about some different types of operators (what they do and where they belong

> in order of operations) later on!

2.2: Modules

Technically, you don't need modules since in theory, you can just write the code yourself. However, Python saves you a lot of time by providing a collection of functions (tools that do tasks if you will) and pre-set values that are packaged in what we call *modules*. Sometimes even modules are grouped together, and these groups of modules are called *packages*.

Python has a substantial list of built-in modules and other things, which you can look at by clicking this link. To use a module that Python knows about, you need to use the `import` keyword. For example, if we wanted to use functions from the `math` module, we would do:

```
import math
```

The `math` module contains a myriad of functions pertaining to, well, math! An example is the `sqrt()` (**sq**uare **r**oo**t** function). You will learn later that you can actually make your own functions, so to help Python know that you specifically want a function from `math`, you must prepend `math.` in front of the function. An example is as shown below:

```
import math

print(math.sqrt(49))
print(math.sqrt(50))
```

```
7.0
7.0710678118654755
```

So what happens if you forget to add the `math.` in front? You just get an error:

```
print(sqrt(49))
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[15], line 1
----> 1 print(sqrt(49))

NameError: name 'sqrt' is not defined
```

2.2.1: `from` keyword

You can specify exactly which function you want with the `from` keyword:

```
from math import sqrt
```

This does not allow you to use any other function from `math` other than `sqrt()`. You can also list multiple functions to import, and of course, you will only be allowed to use those functions:

```
from math import sqrt, pow
```

> ⚠️ **Watch out — `from` requires different syntax!**
>
> Since Python knows exactly which function you want to use, you do not prepend the module in front! An example of the prepending causing an error is shown below:
>
> ```
> from math import sqrt
>
> print(math.sqrt(25))
> ```
>
> ```
> 5.0
> ```
>
> Instead, just call the function directly:
>
> ```
> from math import sqrt
>
> print(sqrt(25))
> ```
>
> ```
> 5.0
> ```

> ⚠️ **Watch out — Using `from` can be dangerous!**
>
> I did mention that you can make your own functions. If they happen to have the same name as a function you imported, some weird (but predictable) behaviors happen. Though not relevant now, do be very wary of using `from`.
>
> There is also the extremely dangerous:
>
> ```
> from math import *
> ```
>
> The `*` means "everything". This means you can access all the functions in `math` (or whatever module you want to import) *without the need to prepend any of those functions*. This is highly discouraged although sometimes I recommend it to students who are doing code that only require the one module anyways just for the sake of speed. However, it is a best practice to use `import` most times.

2.2.2: `as` keyword

Sometimes, you may get a bit tired of typing the `math.` (or `{module}.`) every single time. There is a way to give your module a nickname, and it is with the `as` keyword:

```
import math as m

print(m.sqrt(100))
```

```
10.0
```

2.2.3: Installing your own modules

The community has created its own modules that don't necessarily come standard with Python when you first downloaded it. An example is the `numpy` package, which provides an extensive amount of tools for data science calculations.

Typically, Python comes with `pip`, Python's premier installer. There are arguably "more modern" installers, but `pip` is what Python comes with and works quite fine. If I wanted Python to know of `numpy`, I will type the following command in my terminal:

(Mac, Linux)

```
python3 -m pip install numpy
```

(Windows)

```
python -m pip install numpy
```

> 📝 **Note — If using Windows, make sure Python is on PATH**
>
> Most installation guides on Windows mention to check the box that says something about adding Python to PATH because this allows Python to be immediately accessible by your computer's terminal. If Python does not happen to be on PATH, you can re-install Python and be sure to check that box, or you can manually add Python to PATH yourself (I recommend the former if you're not tech savvy).